

COE 379L Final Project (Project 3)

Music Recommendation System

Jason Lee jl78928

Christopher Deng cmd4923

December 12th, 2025

<https://github.com/leechee/coe379L/tree/main/project3>

INTRODUCTION

We've all been there. Listening to the same song over and over again gets boring, and it's hard to find new, refreshing music. It is especially hard to find music that's similar in key, tempo, and tone. That's why for this project, we want to create an application that recommends songs for you...

Introducing: StrumSense, a music recommendation system perfect for all music lovers!

PROJECT STATEMENT

Our goal for this project is to chain multiple models together with public databases and APIs to create a model that recommends music. This entails creating a pipeline that takes in a song file and extracts features from the song, such as key and tempo. We additionally perform sentiment analysis on the audio to better capture what the song is about. These features are combined into an embedding that represents the nature of the song. Lastly, cosine similarity is computed between songs to find other songs that are the most similar to a selected song. This allows us to have a simple yet relatively robust method of characterizing and finding similar songs.

DATA SOURCES AND TECHNOLOGIES USED

Data Sources

Our primary source of data is the Last.fm dataset. We picked the top 1000 most-played songs from the dataset and primarily used this to construct our library of songs to recommend.

Additionally, the user can input new songs as well, growing the library of songs each time new songs are added. It is important to note that while we use Last.fm as the source of our song files, the embeddings produced by our models for each song are the real dataset that we created and use.

Technologies and Libraries

Python 3.11 – Main development language used for preprocessing, feature extraction, and similarity computation.

Flask – Web framework providing REST API endpoints for audio upload and analysis.

Librosa – Audio analysis library used to extract key musical descriptors, namely tempo and music key using chroma features and beat tracking algorithms.

OpenL3 – A pre-trained deep-learning model (trained on Google's AudioSet) that generates 512-dimensional embedding vectors for musical audio. These embeddings capture semantic content including timbre, rhythm, and harmonic characteristics.

NumPy – Used for vector manipulation, normalization, and cosine similarity calculations between embeddings.

SoundFile – Audio file I/O for loading WAV and MP3 files.

yt-dlp – YouTube audio downloading for building the song database from Last.fm track listings.

Last.fm API – Source of top songs metadata for building the initial database of 100 songs.

requests – HTTP library for testing the API with command-line scripts

METHODS EMPLOYED

While we did not really train or tune the models, we did get a good grasp of how and why these models work. We used Librosa, an audio and music signal processing library, to extract musical features such as tempo and music key, while we used OpenL3 to capture additional audio features such as timbre, intensity, frequency distributions, and emotional qualities. Using these two resources, we are able to build a more robust model by combining their results.

Using Librosa, we can extract music audio features that describe the fundamental properties of a song, such as tempo, beat patterns, harmonic content, and music key. These are all measurable signal properties of the audio, providing much more factual information about the song.

Together, these features allow us to create embeddings for a song's musical foundation and fundamental characteristics. Note that Librosa is a deterministic music audio signal processing library that allows us to extract music features from songs

OpenL3 allows us to interpret the waveforms through various acoustic qualities such as timbre, vocal characteristics, instrumentation, and emotional tone through high-dimensional embeddings. By using OpenL3, we can represent the intangible and often hard-to-categorize emotional qualities in music. We think of it as an audio analog of text sentiment analysis.

Cosine similarity is the final step that allows us to connect all of these extracted features into a meaningful recommendation system. Once Librosa and OpenL3 have each produced their respective embeddings—one describing the musical structure of the song and the other capturing more semantic information—we compare the two songs using cosine similarity. Cosine similarity makes comparisons by using the angle between two embedding vectors to tell how similar two embeddings are. This means that two songs are considered similar if the direction of their feature vectors is aligned.

So to generate final recommendations, we employ a weighted hybrid scoring approach that balances semantic audio similarity with fundamental music features:

$$\text{Final Similarity Score} = (0.70 \times \text{OpenL3 Similarity}) + (0.30 \times \text{Librosa Similarity})$$

The 70-30 weighting reflects the relative importance of each component, as OpenL3 is the primary driver of recommendations, capturing complex acoustic patterns including timbre,

rhythm, harmonic content, and overall sonic texture that are difficult to express through hand-crafted features alone. Librosa provides complementary information about musical structure, ensuring songs match on interpretable dimensions like tempo range, key compatibility, energy levels, and spectral characteristics.

This hybrid approach balances the power of deep learning embeddings with the interpretability of traditional music information retrieval features, giving users both accurate recommendations and clear explanations for why songs are similar.

In practice, cosine similarity lets us measure how close two songs are in terms of both their fundamental musical features and their semantic characteristics. Songs with similar harmonic patterns, vocal textures, energy levels, or emotional tones should theoretically cluster together in this shared embedding space. As cosine similarity is a simple and widely used method for comparing vectors, we believed this was the best way to compute similarity.

Finally, during inference, we just add new songs by running them through the extraction and embedding pipeline. Their embeddings can be immediately compared to the existing dataset without retraining or additional calibration. This allows us to quickly and easily do inference on new songs. However, this does mean that we had to run each of the original 1000 songs through the extraction and embedding pipeline, which may cause issues if we were to drastically increase the number of songs, as the time and computation required as we increase the number of songs is numbered in days (on our personal laptops) while loading thousands of songs.

RESULTS

The overall system successfully extracts audio features and generates recommendations based on musical similarity. The OpenL3 embeddings explain musical semantic characteristics, while Librosa features provide base music features.

Our music recommendation system returns the top 10 most similar songs based on cosine similarity of the model embeddings. We found our similarity scores for the top 10 most similar songs typically range from 0.90 to 0.97, where scores above 0.8 indicate strong acoustic similarity, demonstrating that our system works at least to some degree. There is an inherent

limitation in the fact that we have a limited number of songs, so our similarity scores would only go up as the number of songs increases.

As this is an unsupervised learning task, it is more difficult to evaluate how our system performs compared to a supervised learning problem. Theoretically, songs with similar genres, instrumentation, and energy levels should cluster together naturally in the embedding space, but as our vectors are hundreds of dimensions long, we can't really visualize this. After some user testing, we found that our music recommender does pretty well, with the top recommended song usually having a cosine similarity score of above 0.95, indicating a very similar song. After listening to numerous songs and their respective recommendations, we found our music recommender to work very well and thought the songs were extremely similar to our input song.

Overall, while the system is not fully developed for production-level performance, the results demonstrate that our audio music embeddings provide a strong foundation for content-based music recommendation.

REFERENCES

McFee, B., Raffel, C., Liang, D., et al. "librosa: Audio and Music Signal Analysis in Python." *Proceedings of the 14th Python in Science Conference*, 2015.

OpenL3 Project Documentation. <https://github.com/marl/openl3>

Last.fm API Documentation. <https://www.last.fm/api>

Hershey, S., et al. "CNN Architectures for Large-Scale Audio Classification." *ICASSP*, 2017.

Manning, C., et al. *Foundations of Statistical Natural Language Processing*. MIT Press.